

passgen

Детерминированный генератор паролей

Полное описание программы для защиты проекта

Все возможности, архитектура и реализация

2026

Содержание

1. Идея проекта
2. Как это работает (главный алгоритм)
3. Установка и запуск
4. Интерфейс программы (поля и кнопки)
5. Все функции программы
6. Архитектура кода (app.py и index.html)
7. Описание каждой функции в коде
8. Экспорт и импорт настроек
9. Безопасность

1. Идея проекта

passgen — это программа, которая создает надежные пароли на основе секретной фразы (сида) и названия сервиса. Главная особенность: пароль всегда можно восстановить, зная сид и сервис. Не нужно хранить пароли в базе или в облаке — достаточно запомнить одну фразу.

Программа использует криптографический алгоритм HMAC-SHA256 для детерминированной (то есть всегда одинаковой для одних и тех же входных данных) генерации паролей. Это значит, что на любом устройстве, в любой операционной системе, имея один и тот же сид и название сервиса, вы получите абсолютно одинаковый пароль.

Программа написана на Python (серверная часть) и JavaScript (клиентская часть с интерфейсом HTML/CSS). Интерфейс открывается в отдельном окне браузера без адресной строки, создавая впечатление настоящего настольного приложения.

2. Как это работает (главный алгоритм)

Алгоритм генерации пароля состоит из нескольких шагов:

Шаг 1. Получение ключа

Секретная фраза (сид) преобразуется в байты и используется как ключ для HMAC-SHA256. HMAC (Hash-based Message Authentication Code) — это криптографическая функция, которая гарантирует, что только зная сид, можно восстановить пароль.

Шаг 2. Формирование сообщения

Сообщение состоит из названия сервиса (например, "google"), длины пароля и номера блока. Номер блока увеличивается с каждым шагом, чтобы гарантировать уникальность каждого блока.

Шаг 3. Вычисление HMAC

Для каждого блока вычисляется HMAC-SHA256 от сообщения. Результат — 256 бит (32 байта) случайных данных.

Шаг 4. Преобразование в символы

Из полученных байтов выбираются те, которые меньше максимального значения (чтобы избежать смещения вероятности). Каждый такой байт преобразуется в символ из выбранного набора (заглавные буквы, строчные, цифры, спецсимволы). Процесс повторяется, пока не наберется нужное количество символов.

Шаг 5. Применение дополнительных опций

Если включена опция "Первая заглавная", первый символ пароля делается заглавным. Если включена "Последняя цифра", последний символ заменяется на цифру, вычисленную отдельным HMAC.

Благодаря детерминированности алгоритма, одни и те же входные данные (сид + сервис + настройки) всегда дают один и тот же пароль. Это позволяет не хранить пароли — достаточно помнить сид.

3. Установка и запуск

Для работы программы требуется только Python 3 (любая версия от 3.6 и выше).

На Linux (Arch, Ubuntu и другие):

1. Склонировать репозиторий:

```
git clone git@github.com:inzexg-coder/xxxulay.git
cd xxxulay
```

2. Запустить:

```
python3 app.py
```

На Windows:

1. Скачать репозиторий с GitHub (зеленая кнопка Code -> Download ZIP)

2. Распаковать архив

3. Открыть папку и запустить:

```
python app.py
```

После запуска программа автоматически открывается в отдельном окне браузера без адресной строки. Если у вас установлен Chromium-браузер (Google Chrome, Яндекс.Браузер, Brave, Vivaldi, Edge), то окно будет без панели управления — как настоящее приложение.

4. Интерфейс программы (поля и кнопки)

Интерфейс состоит из карточек (блоков) с полями ввода и кнопками. Все элементы расположены вертикально для удобства использования.

Поле "Сид"

Секретная фраза, которую нужно запомнить. Это главный секрет — без него невозможно восстановить пароль. При вводе сида программа снимает ошибку, если она была показана. Если сид не введен, поле подсвечивается розовой обводкой с сообщением об ошибке. Нажатие Enter в этом поле запускает генерацию пароля.

Поле "Сервис"

Название сервиса, для которого генерируется пароль. Например: google, github, vk, yandex. Можно ввести любое название. Справа от поля есть кнопка "список", которая открывает панель с популярными сервисами (Google, GitHub, VK, Яндекс, Mail, iCloud, Dropbox, Telegram) и ранее сохраненными сервисами пользователя. Каждый сервис в списке отображается с цветным логотипом (кружок с буквой). Клик по сервису заполняет поле. Если сервис не из списка, он автоматически добавляется в список при генерации пароля. Удалить сервис из списка можно правым кликом мыши. Нажатие Enter в этом поле запускает генерацию.

Поле "Длина"

Количество символов в пароле (от 1 до 99). Можно ввести число вручную или использовать стрелки вверх/вниз для изменения. Рекомендуемая длина — от 10 символов.

Набор символов (чекбоксы-кружочки)

Четыре опции, определяющие, какие символы будут в пароле:

- Заглавные: A B C D E F G H I J K L M N P Q R S T U V W X Y Z
- Строчные: a b c d e f g h i k m n p q r s t u v w x y z
- Цифры: 0 1 2 3 4 5 6 7 8 9
- Спецсимволы: ! @ # \$ % & * + = - .

Нужно выбрать хотя бы один набор. Кружочки переключаются кликом.

Расширенные настройки (инверсия)

Дополнительные опции, которые открываются кликом на стрелку "Расширенные":

- Первая заглавная: делает первый символ пароля заглавным
- Последняя цифра: заменяет последний символ на цифру

Эти опции делают пароль более удобным для использования в сервисах, которые требуют заглавную букву и цифру в пароле.

Поле "Логин"

Генерирует простой логин из 6 символов (строчные буквы без путающихся и цифры). Логин детерминирован, но при каждом нажатии кнопки "Создать логин" генерируется новый (используется счетчик). Логин можно скопировать вручную.

Поле "Пароль" (результат)

Сгенерированный пароль отображается в центре экрана моноширинным шрифтом. Он автоматически копируется в буфер обмена при генерации.

Полоска надежности

Под паролем отображается индикатор надежности из трех полосок:

- Розовый (слабый): длина меньше 10 или меньше 3 наборов символов
- Малиновый (средний): длина от 10, 3+ набора символов
- Сиреневый (отличный): длина от 14, все 4 набора символов

Рядом с полосками показываются рекомендации на русском языке.

Кнопки

- "Создать" (основная) — генерирует пароль по текущим настройкам
- "Копировать" — копирует пароль в буфер обмена
- "Создать логин" — генерирует логин (рядом с полем логина)
- "Экспорт" — скачивает JSON-файл с настройками
- "Импорт" — загружает настройки из JSON-файла с выбором сервисов
- "Список" — открывает панель с популярными и сохраненными сервисами

Пресеты

Три быстрых предустановки в выпадающем списке:

- Минимальный: длина 8, буквы + цифры
- Стандартный: длина 10, всё кроме спецсимволов
- Максимальный: длина 16, все типы символов

5. Все функции программы

Ниже перечислены все возможности программы с объяснением, как они работают.

5.1. Генерация пароля по алгоритму HMAC-SHA256

Основная функция программы. Пользователь вводит сид (секретную фразу), выбирает сервис (или вводит свой), настраивает параметры и нажимает "Создать". Программа вычисляет пароль через HMAC-SHA256 с использованием сида как ключа и сервиса + длины как сообщения. Результат детерминирован — для одинаковых входных данных всегда получается одинаковый пароль.

5.2. Сохранение настроек для каждого сервиса

Программа запоминает, какие настройки (длина, набор символов, расширенные опции) были использованы для каждого сервиса. При повторном выборе сервиса из списка или при вводе его названия, настройки автоматически восстанавливаются. Настройки сохраняются в памяти программы на время сессии и не хранятся между запусками (для безопасности).

5.3. Список популярных сервисов с логотипами

В программе есть предустановленный список популярных сервисов (Google, GitHub, VK, Яндекс, Mail, iCloud, Dropbox, Telegram) с цветными кружками-логотипами. При клике на сервис автоматически заполняется поле "Сервис" и подгружаются сохраненные настройки.

5.4. Индикатор надежности пароля

Программа анализирует длину пароля и количество выбранных наборов символов, чтобы оценить его надежность. Три состояния: слабый (розовый, 1 полоска), средний (малиновый, 2 полоски), отличный (сиреневый, 3 полоски). Рядом выводится текстовая рекомендация.

5.5. Генерация логина

Создает простой для запоминания логин из 6 символов. Используются только строчные буквы без путающихся (исключены: i, l, o) и цифры. При каждом нажатии кнопки генерируется новый логин благодаря внутреннему счетчику.

5.6. Автоматическое копирование в буфер обмена

После генерации пароль автоматически копируется в буфер обмена. Появляется сообщение "Скопировано!" на экране. Также есть отдельная кнопка "Копировать" для повторного копирования. Используется современный API буфера обмена (navigator.clipboard).

5.7. Экспорт настроек в JSON

Скачивает файл passgen_export.json со всеми сервисами и их настройками. Файл имеет читаемую структуру с отступами (форматированный JSON).

5.8. Импорт настроек из JSON с выбором сервисов

Загружает ранее экспортированный файл. Программа показывает модальное окно со списком всех сервисов из файла и предлагает выбрать, какие импортировать. Отмеченные сервисы добавляются в список, и их настройки восстанавливаются.

5.9. Сессионный режим (безопасность)

Программа не сохраняет никакие данные на диск. При закрытии окна и повторном открытии все настройки сбрасываются — начинается чистая сессия. Это сделано для безопасности: даже если злоумышленник получит доступ к компьютеру, он не сможет узнать, для каких сервисов и с какими настройками генерировались пароли.

5.10. Анимации интерфейса

Все элементы появляются с плавной анимацией (fadeUp). Кнопки реагируют на нажатие (scale). Панель сервисов и расширенные настройки открываются с анимацией раскрытия. Кружочки чекбоксов анимируются при переключении.

6. Архитектура кода

Программа состоит из двух файлов: app.py (сервер на Python) и index.html (клиент).

6.1. app.py — сервер

Файл app.py запускает простой HTTP-сервер на Python. Он слушает порт 8765 (или следующий свободный) и отдает файлы из папки программы. При запуске сервер пытается открыть окно браузера в режиме приложения (без адресной строки).

Компоненты app.py:

- QuietHandler: класс-обработчик HTTP-запросов. Отличается от стандартного тем, что не выводит логи запросов в консоль, чтобы не засорять терминал пользователя.
- open_app_mode(url): функция, которая пытается открыть браузер в режиме приложения (--app). Проверяет по очереди: Google Chrome, Chromium, Brave, Vivaldi, Microsoft Edge, Firefox. Для Chromium-браузеров устанавливает размер окна 420x620 пикселей. Если ни один браузер не найден, открывает браузер по умолчанию через модуль webbrowser.
- Автоматический выбор порта: если стандартный порт 8765 занят другой программой, сервер автоматически проверяет следующие порты (8766, 8767 и т.д.) пока не найдет свободный.
- Сервер работает в отдельном потоке (daemon), главный поток ожидает нажатия Ctrl+C для корректного завершения работы сервера.

6.2. index.html — клиент

Файл index.html содержит весь пользовательский интерфейс и логику работы. Он состоит из трех частей:

HTML: разметка страницы — карточки (блоки) с полями ввода, выпадающие списки, кнопки. Каждый элемент интерфейса находится внутри своего блока с описанием.

CSS: стилизация в фиолетово-белой гамме с использованием CSS-переменных для цветов. Используются плавные анимации (fadeUp, glow, slideDown, checkPop), градиенты для кнопок, закругленные углы, тени. Полоса прокрутки скрыта для чистоты дизайна.

JavaScript: вся логика программы — от генерации паролей до экспорта/импорта настроек. Использует Web Crypto API для криптографических операций (HMAC-SHA256, SHA-256).

7. Описание каждой функции в коде

Ниже приведено описание всех JavaScript-функций, используемых в программе.

init()

Запускается автоматически при загрузке страницы. Выполняет следующие действия:

- Вызывает loadState() для сброса всех настроек (безопасность — чистая сессия)
- Навешивает обработчик на поле сида: при вводе снимает ошибку
- Навешивает обработчики Enter на поля сида и сервиса для быстрой генерации
- Навешивает обработчик blur на поле сервиса: при потере фокуса обновляет текущий сервис и создает запись настроек, если сервис новый
- Восстанавливает настройки для текущего сервиса или сбрасывает на стандартные
- Отрисовывает список сервисов с логотипами

loadState()

Сбрасывает все данные программы:

- Очищает localStorage от старых сохранений
- Обнуляет объект настроек perSvc
- Очищает массив сервисов svcs
- Сбрасывает текущий сервис на '(empty)'
- Устанавливает стандартные настройки
- Сбрасывает длину на 10 и очищает поле сервиса

Каждый запуск программы начинается с чистого листа.

saveCfg()

Сохраняет текущие настройки (чекбоксы, инверсию, длину) в объект perSvc под именем текущего сервиса (curSvc). Вызывается только при:

- Генерации пароля (функция gen())
- Применении пресета настроек

Это гарантирует, что настройки сохраняются только при реальном использовании.

applyCfg(cfg)

Применяет сохраненные настройки к интерфейсу:

- Восстанавливает состояние всех чекбоксов символов
- Восстанавливает опции инверсии
- Устанавливает длину пароля
- Перерисовывает элементы управления

Вызывается при выборе сервиса из списка

toggleInv(id)

Переключает опции расширенных настроек (инверсию).

Принимает id: 'cap' (первая заглавная) или 'dig' (последняя цифра).

Меняет значение на противоположное (0->1, 1->0) и обновляет отображение кружочка. Не сохраняет настройки мгновенно — сохранение происходит при генерации пароля.

applyPreset()

Применяет готовые пресеты настроек из выпадающего списка:

- Минимальный: длина 8, заглавные+строчные+цифры
- Стандартный: длина 10, заглавные+строчные+цифры
- Максимальный: длина 16, все типы символов

Обновляет чекбоксы и сразу сохраняет настройки для текущего сервиса.

renderChk()

Динамически создает элементы управления для выбора набора символов. Каждый набор отображается как строка с кружочком (чекбокс) и названием. При клике на строку значение переключается, и набор перерисовывается. Наборы символов: заглавные, строчные, цифры, спецсимволы. Каждый набор имеет predetermined список допустимых символов.

renderInv()

Обновляет отображение кружочков для опций инверсии. Просто добавляет или убирает CSS-класс 'on' для соответствующих элементов.

adj(delta)

Изменяет длину пароля на указанное значение (+1 или -1). Проверяет границы: минимальная длина 1, максимальная 99.

toggleSvcPanel()

Открывает или закрывает панель со списком сервисов. Использует CSS-классы для анимации раскрытия/сворачивания. Стрелка рядом с кнопкой поворачивается при открытии.

renderSvcChkips()

Собирает и отображает все сервисы в панели:

- Сначала популярные сервисы, которых нет в сохраненных
- Затем ранее использованные сервисы пользователя

Каждый сервис отображается в виде чипа (кнопки) с логотипом и названием.

chip(item, hasLogo, deletable)

Создает один элемент чипа (кнопку) для сервиса:

- Если hasLogo и сервис из популярного списка: цветной кружок с буквой-логотипом
- Если сервис пользовательский: серый кружок с вопросительным знаком
- При клике: заполняет поле сервиса и применяет настройки
- Если deletable: по правому клику мыши удаляет сервис из списка

Возвращает созданный DOM-элемент.

gen()

Главная функция генерации пароля. Выполняет:

- Проверяет, что сид введен (иначе — розовая подсветка и сообщение об ошибке)
- Проверяет, что выбран хотя бы один набор символов
- Составляет алфавит из выбранных наборов символов
- Вызывает hmacGen() для вычисления пароля
- Применяет опции инверсии (первая заглавная, последняя цифра)
- Отображает пароль в поле результата

- Обновляет полосу надежности
- Копирует пароль в буфер обмена
- Добавляет сервис в список, если его еще нет
- Сохраняет настройки для текущего сервиса

hmacGen(seed, svc, len, alpha)

Криптографическое ядро программы:

- Преобразует сид в байты и создает ключ HMAC через Web Crypto API
- Формирует сообщение: сервис + длина + счетчик (ct)
- Вычисляет HMAC-SHA256 для каждого блока
- Применяет rejection sampling: отбрасывает байты $\geq 256 - (256\% \alpha)$ для равномерности
- Преобразует подходящие байты в символы из алфавита
- Повторяет с увеличением счетчика, пока не наберет нужную длину

Возвращает сгенерированный пароль.

genLogin()

Генерирует логин из 6 символов:

- Использует SHA-256 от строки сид + 'login' + сервис + счетчик
- Счетчик (loginCounter) увеличивается при каждом вызове
- Алфавит логина: строчные буквы без путающихся (a-z без i,l,o) и цифры 2-9
- Берет первые 6 байтов хеша и преобразует в символы

Особенность: каждый вызов дает новый логин (благодаря счетчику).

updateStrength(len, sets)

Анализирует и отображает надежность пароля:

- Слабый (1 розовая полоска): длина < 10 или < 3 наборов символов
- Средний (2 малиновых полоски): длина ≥ 10 и ≥ 3 наборов
- Отличный (3 сиреневых полоски): длина ≥ 14 и $= 4$ набора

Отображает текстовую рекомендацию под полосками.

cpy()

Копирует пароль в буфер обмена:

- Использует современный navigator.clipboard.writeText()
- При ошибке использует устаревший document.execCommand('copy') как запасной
- Показывает сообщение 'Скопировано!' на 1.5 секунды

exportData()

Экспортирует настройки в JSON-файл:

- Собирает все сервисы, для которых есть настройки (perSvc)
- Включает также настройки для '(empty)' если они есть
- Создает объект с полями svcs (список) и perSvc (настройки)
- Форматирует JSON с отступами для читаемости
- Скачивает файл через виртуальную ссылку (click на <a>)

importData(event)

Импортирует настройки из JSON-файла:

- Читает выбранный пользователем файл
- Парсит JSON и проверяет наличие данных passgen
- Если данные корректны, сохраняет их во временную переменную
- Открывает модальное окно выбора сервисов для импорта

showImportModal(d)

Показывает модальное окно со списком сервисов из импортируемого файла. Каждый сервис отображается с чекбоксом (по умолчанию включен). Если в файле нет сервисов, показывает сообщение об ошибке.

confirmImport()

Подтверждает импорт выбранных сервисов:

- Собирает все отмеченные чекбоксы
- Для каждого сервиса загружает настройки в perSvc
- Добавляет сервис в список svcs (если его там не было)
- Сортирует список и перерисовывает
- Закрывает модальное окно
- Показывает сообщение об успехе на 3 секунды

closeImportModal()

Закрывает модальное окно импорта без сохранения. Очищает временные данные импорта.

toggleAdvanced()

Открывает или закрывает панель расширенных настроек. Анимированная высота панели через CSS-свойство max-height. Стрелка поворачивается при открытии панели.

8. Экспорт и импорт настроек

Программа позволяет экспортировать и импортировать настройки в формате JSON.

Формат JSON

Пример структуры файла:

```
{
  "svcs": ["google", "github"],
  "perSvc": {
    "google": {
      "state": {"capitals": 1, "lowercase": 1, "digits": 0, "symbols": 1},
      "inv": {"cap": 0, "dig": 1},
      "len": 16
    }
  }
}
```

Поле svcs — массив названий сервисов.

Поле perSvc — объект, где ключ — название сервиса, значение — объект с настройками:

state — какие наборы символов активны (1/0)

inv — опции инверсии (cap — первая заглавная, dig — последняя цифра)

len — длина пароля в символах

Экспорт

При клике на кнопку "Экспорт" программа собирает все сервисы, для которых были сохранены настройки в текущей сессии, форматирует их в JSON с отступами в 2 пробела и скачивает файл passgen_export.json.

Импорт

При клике на кнопку "Импорт" пользователь выбирает JSON-файл. Программа анализирует файл и показывает окно со списком всех найденных сервисов. Пользователь отмечает нужные сервисы (по умолчанию выбраны все) и нажимает "Импортировать". Настройки загружаются в программу. Если сервис уже существует, его настройки перезаписываются. После импорта показывается уведомление.

9. Безопасность

Программа разработана с учетом следующих принципов безопасности:

Детерминированность

Пароль полностью определяется сидом, сервисом и настройками. При изменении любого параметра пароль меняется кардинально. Это позволяет восстанавливать пароли без их хранения.

Криптостойкость

Используется HMAC-SHA256 — проверенный криптографический алгоритм. SHA-256 устойчив к коллизиям и атакам, используется в банковских системах, блокчейне (биткоин) и протоколах безопасности.

Rejection Sampling

При преобразовании байтов HMAC в символы используется метод rejection sampling: байты $\geq \max$ отбрасываются. Это устраняет смещение вероятности (modulo bias) и гарантирует равномерное распределение символов в пароле.

Сессионный режим

Никакие данные не сохраняются на диск. При закрытии программы все настройки, список сервисов и история генерации безвозвратно удаляются. Каждый запуск — чистая сессия.

Локальность

Вся генерация происходит локально на устройстве пользователя через Web Crypto API браузера. Никакие данные не передаются по сети. HTTP-сервер работает на localhost и доступен только с текущего компьютера.

Минимализм кода

Программа использует только стандартную библиотеку Python (http.server, socketserver) и встроенные возможности браузера (Web Crypto API). Никаких внешних зависимостей — меньше поверхность для атак.